

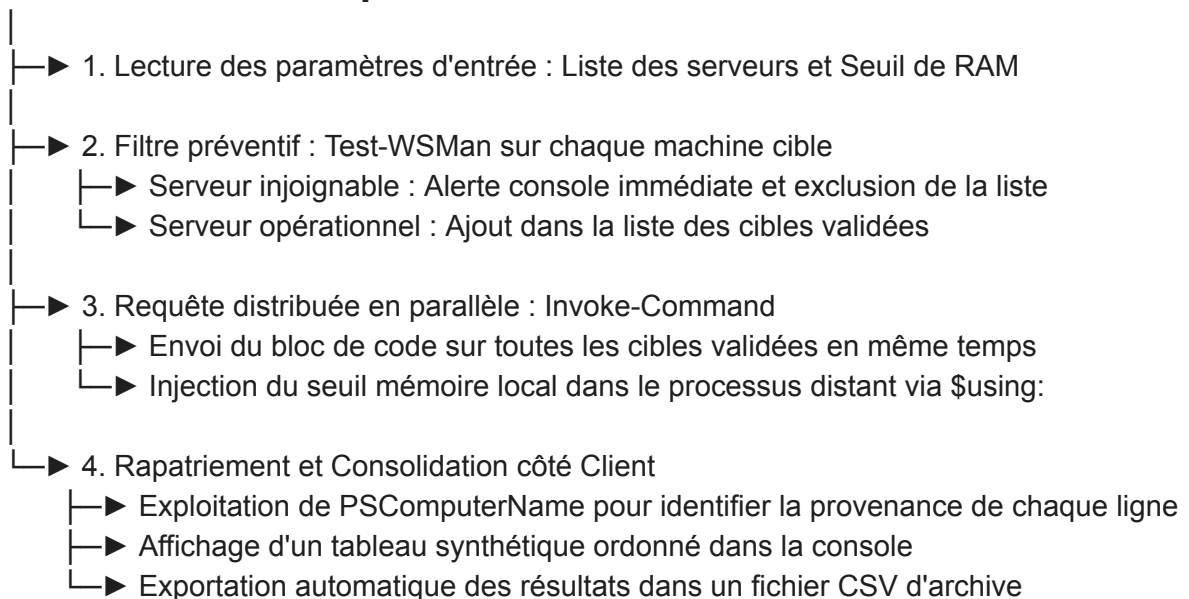
Administration à Distance et Remoting WinRM

L'objectif de ce TP est de concevoir un script d'orchestration centralisé, nommé `audit_parc.ps1`, exécuté depuis votre console d'administration locale, capable d'interroger et de consolider l'état d'un parc de machines distantes de façon simultanée et sécurisée via le protocole WinRM.

1. Schéma Fonctionnel du Script Attendu

Le script `audit_parc.ps1` doit fonctionner selon la façon suivante :

[Poste Client de l'Administrateur]



1.2 Données d'Entrée et Livrables de Sortie

- **Ce que le script reçoit en entrée (Paramètres) :**
 - Une liste de serveurs à auditer (tableau de chaînes de caractères).
 - Un seuil d'alerte de mémoire vive en mégaoctets (nombre entier).
 - Un commutateur optionnel d'exportation CSV.
- **Ce que le script doit afficher dans la console :**
 - Un message d'alerte en rouge pour chaque machine indisponible sur le port WinRM.
 - Un message de progression en cyan annonçant le lancement de la collecte parallèle.
 - Un tableau propre récapitulant les processus suspects, groupés ou triés par machine source.
- **Ce que le script doit produire sur le disque :**

- Un fichier d'inventaire nommé C:\Audit\audit_parc.csv, encodé en UTF-8, contenant l'ensemble des anomalies collectées avec le nom de chaque machine source.

2. Phase Préliminaire : Validation Manuelle des Accès Distants

Avant de lancer l'écriture du script automatisé, réalisez les contrôles préalables unitairements dans votre console locale exécutée avec les privilèges d'administrateur.

2.1 Diagnostic de Connectivité WS-Management

- Interrogez la pile WinRM d'un serveur distant du lab à l'aide de la commande Test-WSMan (sans passer par un ping ICMP).
- Contrôlez l'accessibilité du port d'écoute standard TCP 5985 à l'aide de Test-NetConnection.
- Si vous travaillez en environnement hors-domaine (Workgroup), vérifiez le contenu de la liste TrustedHosts dans le lecteur WSMAN: et autorisez explicitement votre cible.

2.2 Prise en Main Interactive 1-à-1

- Établissez une session distante directe vers votre cible avec Enter-PSSession.
- Observez le changement d'invite de commande (présence du préfixe entre crochets [NomServeur]).
- Exécutez une commande de lecture système (ex: Get-Process ou interrogation du volume C:).
- Quittez la session proprement avec Exit-PSSession pour revenir sur le poste local.

3. Conception du Script d'Orchestration : audit_parc.ps1

Toutes les étapes suivantes doivent être regroupées dans un fichier de script PowerShell nommé audit_parc.ps1.

3.1 Déclaration des Paramètres d'Entrée

- Déclarer l'attribut [CmdletBinding()] en tête de script.
- Déclarer un bloc de paramètres param() comprenant :
 - \$ServeursCibles : tableau de chaînes ([string[]]), acceptant par défaut au moins deux noms ou adresses IP de machines du lab.
 - \$SeuilRAMMo : nombre entier ([int]), définissant la limite de mémoire surveillée (valeur par défaut : 150).
 - \$ExporterCSV : commutateur logique ([switch]).

3.2 Contrôle Préalable et Filtrage des Machines Disponibles

- Créer un tableau vide \$ServeursValides = @().

- Parcourir la collection \$ServeursCibles à l'aide d'une boucle foreach.
- Pour chaque machine :
 - Tester sa disponibilité avec Test-WSMan en masquant les erreurs bloquantes.
 - Si la machine répond : l'ajouter dans la liste \$ServeursValides.
 - Si la machine ne répond pas : afficher une alerte en rouge dans la console indiquant le nom de la machine et l'échec de contact WinRM.

3.3 Collecte Distribuée en Parallèle via Invoke-Command

- Mettre en place une condition vérifiant que \$ServeursValides contient au moins un serveur.
- Exécuter la cmdlet Invoke-Command en lui passant le tableau \$ServeursValides sur le paramètre -ComputerName.
- À l'intérieur du -ScriptBlock exécuté à distance :
 - Récupérer la liste des processus système.
 - Filtrer les processus dont la mémoire de travail (WorkingSet) dépasse le seuil défini sur le client local (obligation d'utiliser le modificateur de portée \$using:SeuilRAMMo).
 - Projeter les propriétés utiles : le nom du processus (Name), son identifiant (Id) et la mémoire arrondie en Mo via une colonne calculée nommée RAM_Mo.
- Stocker le retour global de cette commande dans une variable \$Resultats.

3.4 Consolidation et Restitution des Résultats

- Depuis votre console locale, interroger la variable \$Resultats.
- Mettre en évidence la propriété synthétique PSComputerName générée automatiquement par PowerShell.
- Afficher les résultats sous forme de tableau formaté (Format-Table -AutoSize), en présentant les colonnes suivantes :
 - PSComputerName (Serveur source)
 - Name (Nom du programme)
 - Id (Identifiant PID)
 - RAM_Mo (Consommation en mégaoctets)
- Trier l'affichage par consommation de mémoire décroissante.

3.5 Exportation Optionnelle (Traitement du Switch)

- Si le commutateur -ExporterCSV a été spécifié lors de l'appel :
 - Vérifier la présence du répertoire C:\Audit et le créer si nécessaire.
 - Exporter les objets contenus dans \$Resultats vers C:\Audit\audit_parc.csv.
 - Configurer l'exportation avec un séparateur point-virgule (;), sans métadonnées PowerShell (-NoTypeInfoInformation) et avec un encodage UTF8.
 - Afficher un message de confirmation vert en console indiquant le chemin du fichier généré.

4. Résultats Attendus

4.1 Exemple d'Exécution en Console

```
.\audit_parc.ps1 -ServeursCibles "SRV-DC01", "SRV-FILE01", "SRV-INCONNU" -SeuilRAMMo  
120 -ExporterCSV
```

Sortie console :

ALERTE : Le serveur SRV-INCONNU ne repond pas sur WinRM (Port 5985).
Lancement de la collecte parallele sur : SRV-DC01, SRV-FILE01...

PSComputerName	Name	Id	RAM_Mo
SRV-DC01	lsass	620	284.15
SRV-FILE01	sqlservr	1844	245.80
SRV-FILE01	SYSTEM	4	180.22
SRV-DC01	svchost	912	135.40

Exportation terminee avec succes : C:\Audit\audit_parc.csv

4.2 Extrait du Fichier Produit (C:\Audit\audit_parc.csv)

```
"PSComputerName","Name","Id","RAM_Mo"  
"SRV-DC01","lsass","620","284.15"  
"SRV-FILE01","sqlservr","1844","245.80"  
"SRV-FILE01","SYSTEM","4","180.22"  
"SRV-DC01","svchost","912","135.40"
```